Learned Page Replacement in xv6

Week 15 update: new workloads, a wider model search, and the kernel

Summary

Week 14 showed that a small linear model over signals the kernel can see (accessed and dirty bits and cheap bookkeeping) beats Clock by 10–15% in faults on our benchmarks. This week tested how far that result reaches. All numbers are faults relative to Clock (below 1 is better), averaged geometrically over memory sizes of 5, 10 and 20% of each program's pages.

- **More, harder workloads.** Added the fixed `lzw` benchmark to the full pipeline, a synthetic pattern suite with known answers (`patbench`), replays of real Twitter cache traces, real Redis and SQLite page traces, and four application-shaped programs (pointer chase, sparse matrix–vector product, hash join, Bloom filter). Every program's access string is checked reference by reference against an independent model.
- **Literature baselines.** ARC, S3-FIFO, an MGLRU analogue, LHD and exact MRU are implemented in the simulator and checked against independent reference implementations. LHD is the strongest untrained policy (0.90–0.93 on our data); ARC and S3-FIFO are mixed, and S3-FIFO is worse than Clock on graph, matmul and sort.
- **Wider model comparison.** Trees (CART, random forest, boosted trees, ranking boosted trees, logistic regression) and neural nets (deeper MLP, set-attention, CNN and LSTM over page history, a CNN over the kernel's own scan-bit history, mixed models) were fitted on every scope and selected on validation only. With signals the kernel can actually observe, **no model class beats the linear scorer by the 2% and at-most-2×-cost bar overall**. Gains of 5–12% appear on single workloads (kv, the Twitter replays); models given oracle information gain up to 6% in distribution and fail badly on the real Redis and SQLite traces.
- **In the xv6 kernel** (157 runs on the new workloads, plus 61 with sampled scoring): the learned policy cuts faults by 16% on lzw (0.84), 15% on the Twitter replays (0.85) and 15% on patbench (0.85) against Clock, and does nothing on the application programs (1.00). The simulator predicts kernel faults to within about 4%, with one unexplained exception (Section 6). Sampled scoring, now in the kernel, roughly halves the selection cost for 1–2.5% more faults.
- **Five extra models were ported to the kernel and their time delay measured** (Section 7). On kv a CNN over the aging bits and a deeper MLP are about 2.5% better than the linear model, but cost 85 and 148 times its instructions per eviction (1.7 and 2.9 million instructions against 19 000); boosted trees equal it at 21 times; on graph the linear model is clearly better. Timing is repeatable to under 0.3% because QEMU's clock is driven by instruction count.
- **A reproducibility bug was found and fixed.** Training samples depended on Python's per-process string hashing, so datasets could not be rebuilt identically. All models were refit; Clock, LRU and the other classical results are unchanged.

1 What changed in the data and the pipeline

Reproducibility fix. The sampling seed of each training stream used Python's `hash()`, which is randomised per process, so two builds of the same dataset differed. The seed is now a CRC32 of the stream name, a capacity and an optional salt. Rebuilding twice is bit-identical (checked over 11 dataset files); all 4 272 classical result rows of the original streams are unchanged. Because the old caches could not be recovered, every model was refit, so numbers move slightly from week 14 (for example the global default kernel model now has five features instead of four). The noise floor from sampling alone is measured in Section 8.

lzw. The corrected lzw benchmark from the team (`lzw-r20` for training, validation and test; `lzw-r30` held out) now runs through the whole pipeline: simulator sweep, neural models, selection, test, kernel export.

2 New workloads

workload	what it does	why it is in the study
`patbench`	11 synthetic modes over 1 024 pages: cyclic loop, scan with a hot set (two sizes), Zipf with skew 0.6–1.2, uniform, hot set that moves (two speeds), loop/Zipf switching	Known answers: a loop defeats LRU, FIFO and Clock and is solved by MRU; `uniform` has nothing to learn, so any gain there is a leak
`kv-T` (Twitter)	kvbench driven by real key streams from six production cache clusters, keys hash-sampled to kvbench's 1 024-key space (4 700–40 000 operations)	Real popularity and write mix; whole clusters are held out
real traces	Redis and SQLite page-touch strings collected on a host (12.3M and 8.6M references, no read/write marks)	Programs we did not write; tests transfer to a different kind of recording
`appbench`	pointer chase (mcf-like), CSR sparse matrix–vector, hash join, Bloom filter, each with an explicit multi-array layout	Irregular, scan-plus-random structure the other programs lack

Table 1: Workloads added this week. All are held out of training: they only test.

How strings are validated. Each program's reference string has an exact host-side model. The kernel's trace equals the model line for line for all 11 patbench modes (24 short runs and 9 full runs), and for all four appbench modes (30 000 references each, with matching read, write and checksum counters). The remaining production traces were then generated from the model rather than by booting xv6 for 17–45 minutes each; their logs carry a marker saying so and no kernel fault counters. The Twitter streams and the real traces are not generated: kvbench ran them in the kernel, and the Redis/SQLite files are the host recordings.

workload	variants	streams	references	held-out streams
app	4	12	12.0M	12
btree	5	25	7.1M	5
graph	4	14	77.9M	3
kv	13	41	8.5M	11
lzw	2	12	61.4M	6
matmul	6	6	13.8M	2
pat	11	55	55.0M	10
redis	1	1	12.3M	1
sort	3	9	16.6M	3
sqlite	1	1	8.6M	1

Table 2: The study dataset after this week (176 streams).

3 Literature baselines

Implemented in the simulator and cross-checked against independent reference implementations on 475 cases (identical counts): ARC (integer adaptation), S3-FIFO (as published, and a form that reads accessed bits only at queue heads), an MGLRU analogue for anonymous memory (generations, aging walk, rescue at eviction), LHD in a scan form, and exact MRU. MRU equals Belady on cyclic loops (9 of 9 checks), and LRU, FIFO and Clock miss every access of a loop larger than memory, as theory says. Table 4 shows them next to the learned models.

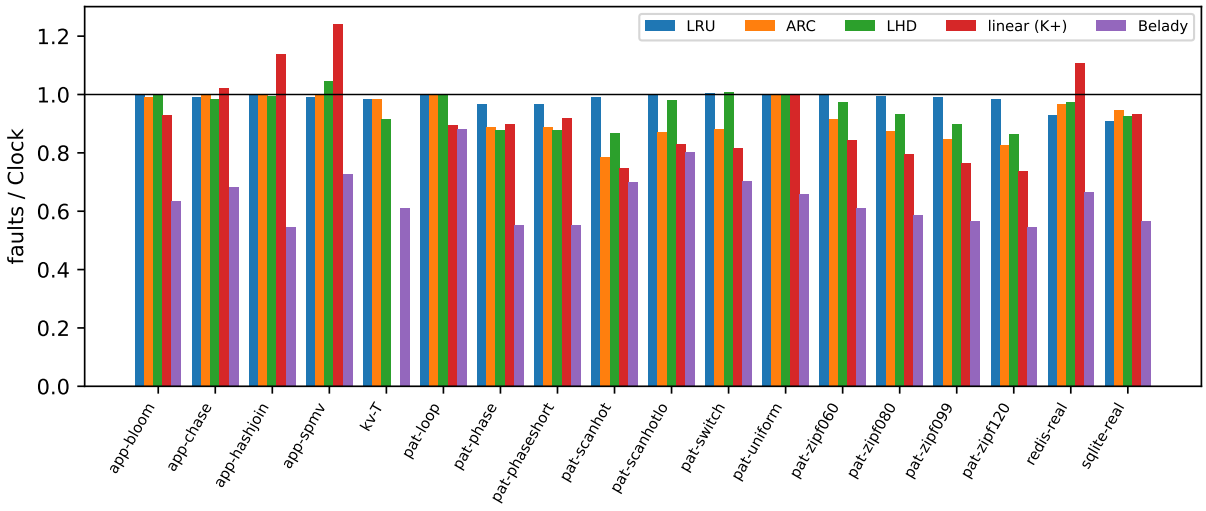


Figure 1: Faults relative to Clock on workloads no model was trained on. Linear (K+) is the global linear model over all kernel signals.

4 Transfer to unseen workloads

- **patbench known answers hold.** On `loop` FIFO, LRU, Clock, ARC and LHD miss everything and MRU matches Belady; on `uniform` every online policy is within 0.0003 of $1 - C/P$, so `uniform` is a clean leak control: no model gains there.
- The linear kernel-signal model beats ARC, S3-FIFO and LHD on the skewed and scan-plus-hot patterns (for example 0.76 against 0.85, 0.81 and 0.90 on Zipf 0.99) and on the Twitter replays (in the kernel, Section 6).
- **Where it loses.** Moving-hot-set patterns, SpMV and hash join. The oracle-feature variant (F+K) raises faults 2.6-fold on the moving hot set and reaches the 3×-Clock cap on Redis. The global kernel-signal linear model is 24% and 14% worse than Clock on SpMV and hash join. Real Redis/SQLite traces carry no write information, so dirty-bit signals are absent there, and LRU (0.93, 0.91) is the best simple policy.

stream	LRU	ARC	S3-FIFO	LHD	Belady	linear K+	linear F+K	linear kv K+
app-bloom	1.000	0.989	0.962	0.998	0.635	0.930	0.926	–
app-chase	0.991	1.002	1.021	0.984	0.683	1.021	1.020	–
app-hashjoin	0.998	1.000	1.012	0.994	0.544	1.138	1.017	–
app-spmv	0.992	1.000	1.026	1.046	0.727	1.239	0.995	–
kv-T	0.983	0.983	1.028	0.916	0.611	–	–	0.980
pat-loop	1.000	1.000	1.000	1.000	0.882	0.894	0.887	–
pat-phase	0.968	0.887	0.899	0.876	0.552	0.899	2.644	–
pat-phaseshort	0.967	0.888	0.909	0.879	0.552	0.918	2.686	–
pat-scanhot	0.992	0.786	0.795	0.866	0.700	0.748	0.744	–
pat-scanhotlo	1.001	0.870	0.874	0.982	0.803	0.829	0.903	–
pat-switch	1.005	0.882	0.877	1.007	0.704	0.817	0.805	–
pat-uniform	1.000	1.000	1.000	1.000	0.658	1.000	1.000	–
pat-zipf060	0.998	0.915	0.881	0.975	0.612	0.845	0.824	–
pat-zipf080	0.994	0.874	0.837	0.932	0.587	0.796	0.773	–
pat-zipf099	0.990	0.846	0.806	0.897	0.565	0.764	0.737	–
pat-zipf120	0.984	0.825	0.783	0.863	0.544	0.737	0.710	–
redis-real	0.929	0.967	1.104	0.973	0.667	1.106	3.000	–
sqlite-real	0.908	0.947	1.082	0.927	0.567	0.934	1.318	–

Table 3: Faults / Clock per stream group (geometric mean over streams and the three memory sizes). Simulator results; the three last columns are linear models (global kernel signals, global oracle+kernel signals, kv-scope kernel signals).

5 Wider model comparison

Protocol. The grid was written before any result: kernel-observable inputs (K) or all inputs (FK), with or without candidate-set rank features; CART (depth 4, 8), random forest, boosted trees (30 and 100 trees, 8 and 16 leaves), ranking boosted trees, logistic regression; eight neural architectures. Candidate sets come from the same recorded eviction decisions as before; the target is the log of the next-use distance. Probation is fixed at 2 scans. Selection is on validation streams at 10% memory only; the chosen models are then replayed on test, held-out and transfer streams at 5, 10 and 20%. Intervals are 95% stream-level bootstraps. The neural models run as numpy callbacks in the simulator (checked against PyTorch to 10^{-6}) and are not run on streams of more than 400 000 references for cost; the C tree scorer is checked against scikit-learn and LightGBM to 3×10^{-6}. A follow-up added four- to sixteen-tree boosted ensembles to meet the cost bar.

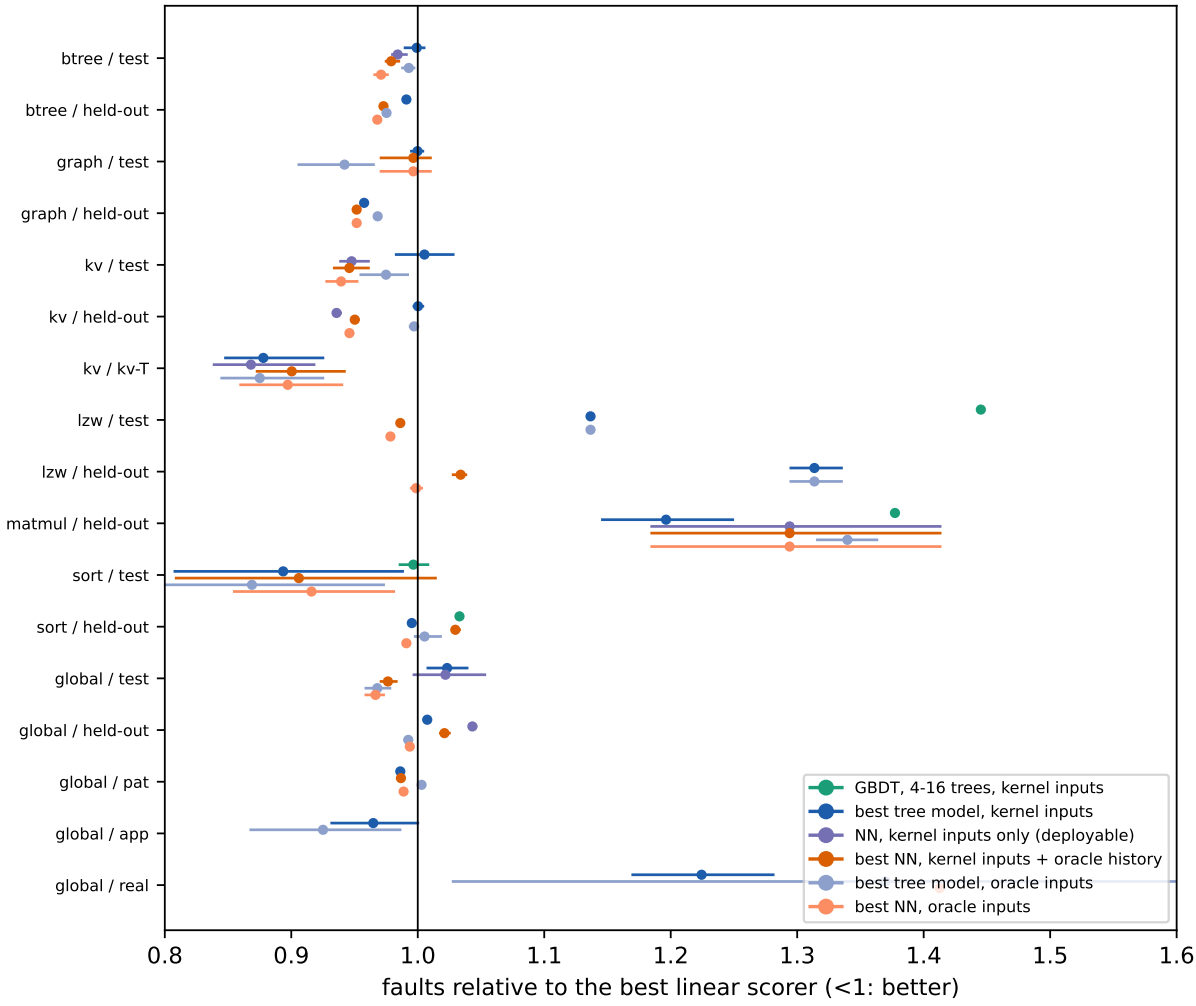


Figure 2: Faults relative to the best linear scorer (below 1: the model is better), with 95% intervals. Models with oracle inputs (blue-grey, orange-red, "oracle") cannot run in a kernel.

What the wider comparison says.

- **Richer models help only where they are given information a kernel does not have.** With oracle inputs the best tree and neural models are 2–5% better than the best linear model in distribution; with kernel inputs only, trees tie the linear model (btree 0.834 against 0.835, graph held-out 0.765 against 0.798, global 0.890 against 0.870) and small boosted ensembles (4–16 trees) are *worse*: lzw 1.19 and 1.57 against 0.82 and 0.81 for the linear model.
- The deployable neural nets (deep MLP, set-attention, aging-bit CNN) gain 5–6% on kv and 2% on btree and lose 2–4% globally. Their kv gain is the one result that passes the 2% bar on a workload scope, at roughly 100× the arithmetic of a linear scorer.
- **Robustness is the real difference.** On the Twitter and application streams the kernel-input trees

scope	split	LRU	ARC	S3F	LHD	linear K	tree K	NN K	NN K†	tree FK	NN FK	Belady
btree	test	0.994	0.950	0.862	0.948	0.835	0.834	0.822	0.817	0.829	0.811	0.550
btree	held-out	0.997	0.975	0.908	0.974	0.875	0.867	–	0.851	0.853	0.847	0.615
graph	test	0.990	0.991	1.161	0.865	0.847	0.847	–	0.844	0.806	0.844	0.456
graph	held-out	0.992	0.993	1.256	0.816	0.798	0.765	–	0.760	0.742	0.760	0.482
kv	test	0.984	0.986	1.047	0.906	0.897	0.902	0.850	0.849	0.875	0.843	0.559
kv	held-out	0.982	0.982	1.013	0.901	0.902	0.903	0.845	0.858	0.900	0.854	0.577
lzw	test	0.997	0.968	0.889	0.892	0.821	0.933	–	0.809	0.933	0.803	0.466
lzw	held-out	0.981	0.957	0.866	0.887	0.813	1.068	–	0.841	1.068	0.812	0.409
matmul	held-out	0.879	0.875	1.508	1.036	0.671	0.803	0.868	0.868	0.899	0.868	0.617
sort	test	1.012	1.013	1.321	1.008	0.983	0.879	–	0.891	0.854	0.901	0.830
sort	held-out	0.999	0.999	1.084	1.019	0.933	0.928	–	0.961	0.938	0.924	0.896
global	test	0.988	0.970	0.960	0.925	0.870	0.890	0.889	0.850	0.842	0.841	0.555
global	held-out	0.982	0.982	1.013	0.901	0.891	0.898	0.930	0.910	0.885	0.886	0.577

Table 4: Faults / Clock on test and held-out streams. "linear K" is the week-14 model class (kernel signals); "tree K" and "NN K" use kernel signals only; "NN K†" also reads the oracle reuse history; "FK" adds oracle inputs. A dash: not run (cost rule) or not selected.

beat the linear model on the Twitter replays by 12% (0.86 against 0.98) and on the application streams by 3.5% (1.04 against 1.08), because they do not extrapolate the way a linear score does; but on the real Redis and SQLite traces every complex model is 22–41% worse than the linear one. None of the extra capacity is free of this trade.

- matmul has no validation streams (selection used its training streams), and every richer model is 20–30% worse than the linear one on its held-out streams: more capacity overfits when there is little data.

6 In the xv6 kernel

The refit linear models were re-exported to integer weights and the learned policy ran in the real kernel on the new workloads (patbench and appbench for 50 000 accesses; lzw and the Twitter replays in full) at 10% memory, all policies, same frame counts as the simulator.

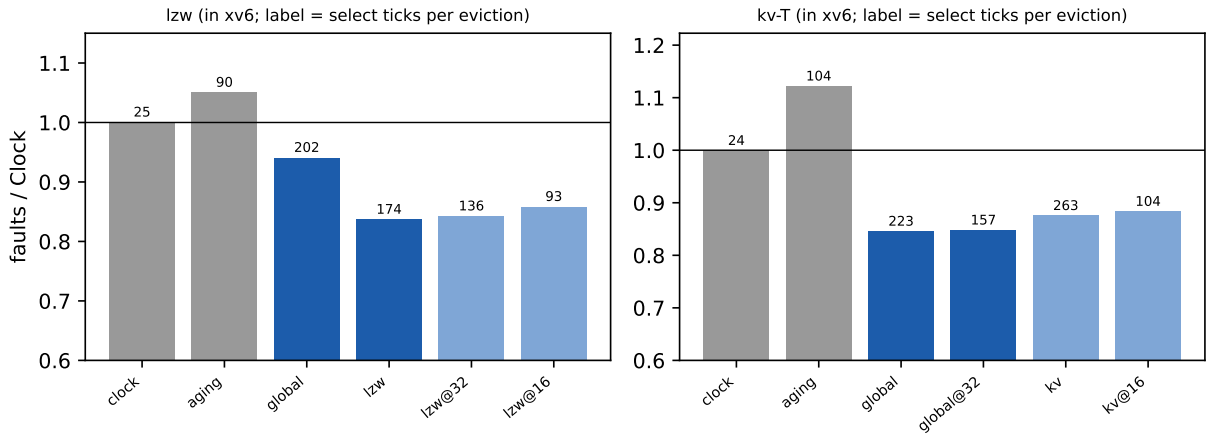


Figure 3: Kernel faults relative to Clock; the number on each bar is the selection cost in timer ticks per eviction (Clock: 24). "@k" scans and scores only k random candidates per eviction.

- **The simulator predicts the kernel.** Kernel over simulator faults, geometric mean per policy: 0.98 on lzw, 0.96–0.98 on patbench and appbench (prefix replay), and 0.98–1.01 for the Twitter replays with the global and kernel-only models. The differences come from the kernel's own pages (code, stack) and capacity cliffs.
- **One exception, not explained.** For the kv-scope linear model on the Twitter replays the kernel is about 10% better than the simulator (0.90; kernel 0.88 of Clock against 0.98 in the simulator). My first guess, that the replay program reads its key file through a buffer the simulator's trace does not contain, is contradicted by the later runs: the CNN, MLP and boosted-tree models agree with the simulator to 1% on the same replays. The cause is specific to that one linear model and is still open.

workload	policy	runs	faults / Clock	select ticks / eviction
lzw	fifo	7	1.132	8
lzw	clock	7	1.000	25
lzw	aging	7	1.050	90
lzw	lfu	7	1.060	90
lzw	ml global	7	0.941	202
lzw	ml lzw	7	0.837	174
lzw	ml lzw@64	7	0.837	173
lzw	ml lzw@32	7	0.842	136
lzw	ml lzw@16	7	0.858	93
kv-T	fifo	6	1.315	8
kv-T	clock	6	1.000	24
kv-T	aging	6	1.123	104
kv-T	lfu	6	1.180	102
kv-T	ml global	6	0.846	223
kv-T	ml global@32	6	0.849	157
kv-T	ml kv	6	0.876	263
kv-T	ml kv@32	6	0.875	153
kv-T	ml kv@16	6	0.884	104
pat	fifo	7	1.195	10
pat	clock	7	1.000	22
pat	aging	7	1.143	158
pat	lfu	7	1.148	159
pat	ml global	7	0.853	428
pat	ml global@32	7	0.861	182
pat	ml global@16	7	0.873	117
pat	ml global-k	7	0.950	370
app	fifo	4	1.221	10
app	clock	4	1.000	24
app	aging	4	0.983	133
app	lfu	4	1.092	134
app	ml global	4	1.004	308
app	ml global@32	4	0.996	148
app	ml global@16	4	0.992	103
app	ml global-k	4	0.997	273

Table 5: In-kernel runs (faults = zero-fill plus swap faults; geometric means over streams).

- The learned policy saves 16% of faults on lzw (per-workload model; 6% for the global model), 15% on the Twitter replays and 15% on patbench, and does not help on the application programs.
- **Cost, and sampled scoring.** The full scorer costs 170–430 ticks per eviction against 24 for Clock (7–17×). Scanning and scoring a random sample of 32 candidates takes 40–80% of that (136 ticks on lzw, 157 on the Twitter replays, 182 on patbench) for 0–1% more faults; 16 candidates takes 30–55% for 1–2.5%. The sampler is a partial Fisher–Yates over the candidate array and needs no extra memory.

7 The five extra models in the kernel

The extra models that use only kernel-visible signals were ranked on validation data (no test data used). The best of each of five kinds were converted to whole-number arithmetic and put into the kernel as built-in models, chosen with `vmrun ml <name>`: a CNN over the aging bits (trained on kv), set-attention (graph), a deeper MLP (kv), ranking boosted trees (graph) and boosted trees (kv). The integer versions choose the same victim as the original models on 381–399 of 400 recorded decisions, and give the same faults in the simulator to within 0.5% (one exception: ranking trees on one graph stream, 13% more). In xv6 they were run on every kv and graph test and held-out stream and the Twitter replays at 10% memory.

Time delay. The kernel now counts, per eviction, the time to build the candidate list, to choose the victim and for the whole eviction without the wait for the swap-out write. QEMU is run in a mode where its clock advances with the instructions executed (`QEMU_ICOUNT=1`), so the numbers do not depend on how busy the host is: two runs of the same policy differ by under 0.3% (0% for most). One run at a time, same stream and memory size for every policy.

policy	kv test	kv heldout	kv-T	graph test	graph heldout
FIFO	1.280	1.310	1.315	2.096	1.289
Aging	1.106	1.133	1.123	1.585	1.264
linear, global	0.874	0.885	0.846	0.946	0.573
linear, kv	0.871	0.878	0.876	–	–
linear, graph	–	–	–	0.618	0.541
CNN on aging bits (kv)	0.849	0.850	0.853	–	–
deeper MLP (kv)	0.853	0.848	0.860	–	–
boosted trees (kv)	0.871	0.903	0.853	–	–
set-attention (graph)	–	–	–	1.145	0.917
ranking trees (graph)	–	–	–	0.697	0.676

Table 6: Faults relative to Clock in xv6 (geometric mean over streams). Each new model is shown only on the workload it was trained for.

policy	instructions to choose	x linear (kv)	instructions, whole eviction
FIFO (floor)	327	0.02	6,658
Clock	513	0.03	6,744
Aging	6,870	0.35	13,250
LFU	6,776	0.35	13,182
linear, global	18,512	0.95	24,650
linear, kv	19,422	1.00	25,538
linear, kv, 32 sampled	11,542	0.59	17,654
linear, kv, 16 sampled	5,824	0.30	11,968
CNN on aging bits	1,658,487	85.39	1,664,638
deeper MLP	2,868,034	147.67	2,874,219
boosted trees	416,962	21.47	423,127
set-attention	1,523,492	78.44	1,529,675
ranking trees	410,089	21.11	416,099

Table 7: Instructions per eviction on one kv stream (57 candidates per eviction). FIFO scans nothing, so it is the floor: the timer reads and the surrounding eviction code. At one nanosecond per instruction, 1 658 487 instructions is 1.7 ms.

- **On kv, the CNN and the MLP are about 2.5% better than the linear kv model** (0.849 and 0.853 against 0.871 on test; 0.850 and 0.848 against 0.878 held-out), less than the 5–6% the simulator predicted, and cost 85 and 148 times the linear model's instructions to choose. The boosted trees equal the linear model (0.871) at 21 times the cost.
- **On the Twitter replays** the three kv models are 0.853–0.860 against 0.876 for the linear kv model and 0.846 for the global linear model, so the global linear model is as good as the expensive ones there.
- **On graph the linear model wins clearly**: 0.618 (test) and 0.541 (held-out) against 0.697 and 0.676 for the ranking trees and 1.145 and 0.917 for set-attention, which is worse than Clock on test.
- **Kernel against simulator.** For the CNN, MLP and trees the kernel agrees with the simulator to 0.4–2.4% on kv and to 1% on the replays, so the conversion and the kernel code are right. On graph the kernel and simulator disagree more (ranking trees 1.06, set-attention 1.59 times the simulator's faults), but graph is noisy for every policy (Clock itself is 0.85 times its simulated faults) because of capacity cliffs. The poor set-attention result in the kernel is not explained.
- **Cheaper linear scoring is the better trade.** Scoring 16 sampled candidates costs 0.3 times the full linear model's instructions for 1–2.5% more faults.

8 Noise floor

The linear model was refit from three extra datasets built with different sampling seeds and replayed on the same test and held-out cells. The faults/Clock geo-mean of the global K+K+ and F+K models for each data seed is

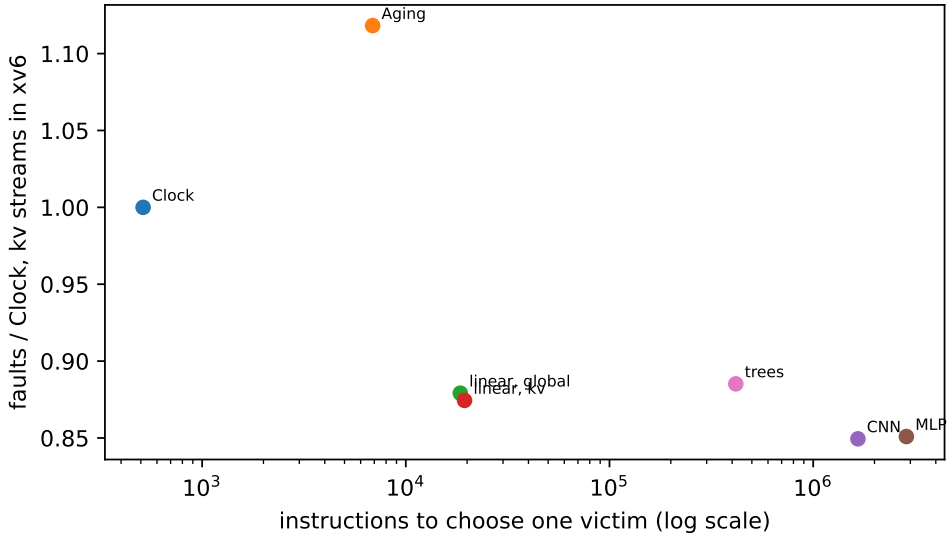


Figure 4: Fault reduction against the cost of choosing a victim on the kv streams.

feature group	faults/Clock per data seed	range
F+K	0.9452 / 0.9433 / 0.9444 / 0.9459	0.0026
K+K+	0.8942 / 0.8942 / 0.8945 / 0.8948	0.0005

Data sampling moves the result by at most 0.0005 (kernel signals) and 0.0026 (with oracle signals), under 0.3%, so the differences between models reported above (1–14%) are not artefacts of which training rows were drawn. It does not measure the other sources of variance (the choice of streams, which the bootstrap intervals cover).

9 Limits and what is next

- The real Redis/SQLite traces and the Twitter replays are small or lack read/write marks; the Twitter streams are hash-sampled to 1 024 keys, which preserves each key's request sequence but not the original working-set size.
- Neural models were not scored on 1M+ reference streams, so the patbench and appbench groups compare trees only.
- The kernel runs of patbench and appbench use 50 000 accesses, not the full 1M, for time.
- Selection cost is measured in instructions on one kv stream with 57 candidates per eviction; it grows with the number of candidates, which was not varied. The timing floor is FIFO, not a separate empty policy.
- Next: add a heap/allocator-churn program; explain the kv-scope linear model's gap on the replays and the set-attention gap on graph; measure cost against the number of candidates.

Reproducing

```
python3 tools/ml2/extra_trees.py fit | select | test      # tree and neural models (extra_nn.py fit first)
python3 tools/ml2/extra_report.py                          # extra models vs linear vs baselines
python3 tools/ml2/exp_transfer.py run [--set real|app]   # transfer streams
python3 tools/ml2/kernel_eval.py --only lzw,kv-T --fracs 0.1  [--models lzw@32]
python3 tools/ml2/kernel_vs_sim.py --only lzw,kv-T,pat,app --tag _w15
python3 tools/ml2/report_w15.py                           # tables and figures of this report
```